Image classification with convolutional neural networks

Deep Learning

Autorzy:

Aleksandra Kwiatkowska

Małgorzata Mokwa

31 marca 2026

Abstract

Image classification remains a central problem in computer vision, driven by the rapid advancement of deep learning methods, particularly Convolutional Neural Networks (CNNs). This work presents a comparative study of multiple neural network architectures, ranging from a simple Multilayer Perceptron (MLP) to more sophisticated convolutional models and lightweight pretrained networks. Special attention is given to the role of convolutional operations in capturing spatial hierarchies and improving generalization performance.

We systematically evaluate the impact of architectural choices, optimization strategies, regularization techniques, and data augmentation on classification accuracy using the CINIC-10 dataset. Furthermore, we explore transfer learning through pretrained models and analyze the effectiveness of progressive layer unfreezing during fine-tuning.

To assess robustness, experiments are also conducted under limited data conditions, highlighting the advantages of convolution-based and pretrained approaches in low-data regimes. The results demonstrate that convolutional architectures, particularly those leveraging pretrained representations, significantly outperform fully connected baselines, confirming their effectiveness in modern image classification tasks.

Keywords: Image Classification, CNN, Data Augmentation, Few-shot Learning

Table of Contents

1 Research problem **2**

2 Theoretical introduction and literature review **2**

3 Conducted experiments **3**

3.1 Architectural Comparison 3

3.1.1 Mulit Layer Perceptron (MLP) 4

3.1.2 Convolutional Neural Networks (CNN) 6

3.1.3 MobileNetV2 8

3.1.4 ResNet-18 10

3.2 Subset Training 13

3.3 Few-Shot Learning 15

4 Conclusions **17**

1 Research problem

Image classification is one of the fundamental tasks in computer vision, aiming to assign a label to an input image based on its visual content. This report focuses on the comparative analysis of different neural network architectures for image classification on the CINIC-10 dataset. In particular, we evaluate the performance of a Multilayer Perceptron (MLP), a simple Convolutional Neural Network (CNN), and more advanced pretrained architectures such as MobileNetV2 and ResNet-18.

The study investigates how architectural design, training-related hyperparameters, regularization techniques, and data augmentation strategies influence model performance. Additionally, experiments are conducted under reduced training data settings to assess model robustness in few-shot scenarios.

2 Theoretical introduction and literature review

Over the years, many types of neural networks have been proposed to solve the image classification problem, ranging from simple fully connected networks to complex deep architectures. One of the earliest approaches is the Multilayer Perceptron (MLP), which consists of layers of neurons fully connected to the previous layer. MLPs can approximate complex functions and learn from labeled data [5], but when applied to images, they often perform poorly.

Convolutional Neural Networks (CNNs) were introduced to address these limitations and are specifically designed for image data [4]. CNNs use convolutional layers to extract features directly from the image, preserving spatial relationships between pixels. By learning hierarchical representations—from simple edges in early layers to complex object parts in deeper layers—CNNs achieve much better performance on visual tasks than MLPs. Pooling layers, such as max pooling, reduce spatial dimensions and make the network more robust to translations.

Deep CNNs face training difficulties due to vanishing gradients. Residual Networks (ResNets) tackle this issue by introducing skip connections, allowing the network to learn residual functions rather than direct mappings [1]. This innovation makes it feasible to train extremely deep models, with ResNet-18, ResNet-34, and even deeper versions

achieving top results on many different classification datasets.

MobileNet is another influential architecture designed for resource-constrained environments such as mobile devices [2]. It employs depthwise separable convolutions, which drastically reduce the number of parameters and computational cost while maintaining high accuracy. MobileNet is often used in transfer learning setups, where pretrained weights from large datasets like ImageNet are fine-tuned on smaller, task-specific datasets.

While MLPs, CNNs, ResNet, and MobileNet are widely used and studied in image classification, they are far from the only available options. Other worth mentioning architectures include VGG [6], DenseNet [3], and Vision Transformers (ViT) [**dosovitskiy2020image**]. The choice of architecture typically depends on factors such as dataset size, computational resources, and the trade-off between accuracy and efficiency.

3 Conducted experiments

3.1 Architectural Comparison

The objective of this study is to systematically evaluate the impact of different neural network architectures, training-related hyperparameters (learning rate and optimizer), regularization techniques (dropout rate and weight decay) and data augmentation methods (rotation, color jitter, gaussian blur, MixUp, edge-enhanced - sobel filter) on image classification performance.

For each considered architecture, experiments were conducted following a consistent and controlled procedure. We began with a common baseline configuration:

- learning rate = 0.1
- optimizer = SGD
- dropout rate = 0.3
- weight decay = 10^{-4}

Starting from this setup, hyperparameters were tuned sequentially. Specifically, we employed a greedy search strategy: each parameter was varied independently while keeping the others fixed, and the best-performing value was selected based on test set performance.

In addition, we also analyzed training dynamics, including training loss stability and signs of overfitting observed on the validation set. Once the optimal value for a given parameter was identified, it was fixed, and the next parameter was tuned.

After determining the best combination of training-related and regularization-related hyperparameters, we proceeded to evaluate the impact of different data augmentation techniques.

In the following sections, we present a detailed analysis of results for each network architecture separately. Finally, we provide a comparative discussion summarizing the performance differences across all evaluated models.

3.1.1 Mulit Layer Perceptron (MLP)

Firstly, we performed experiments on training-related hyperparameters, the results of which are shown in Figure 1. We observed that too low learning rate values result in very slow training, so a **learning rate of 0.01** was selected as the best, 0.1 rate has slightly better results, but training was unstable and might caused overfitting. Among the tested optimizers, the best and most stable results were achieved using **SGD with momentum**.

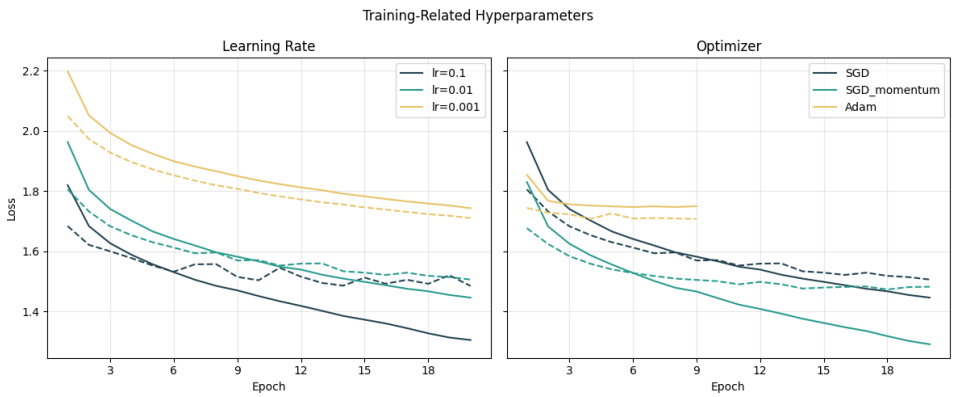


Figure 1: Training and validation loss curves for the MLP model under different training hyperparameters.

During the evaluation of regularization-related hyperparameters, we analyzed the impact of different dropout rates on model performance. We observed that an appropriately chosen dropout rate helps prevent overfitting. Both values 0.3 and 0.5 have similar effects, **dropout rate of 0.3** was chosen. As shown in Figure 2, excessively high weight decay values caused the model to stop learning effectively. 10^{-5} **weight decay** was selected for further experiments.

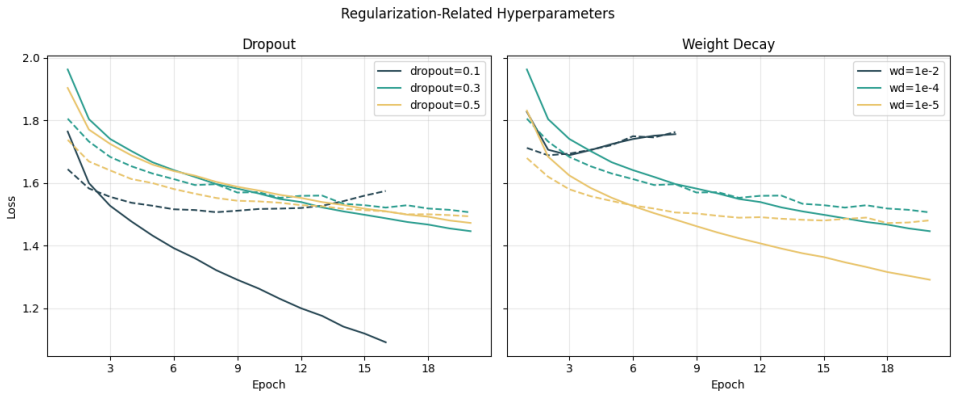


Figure 2: Training and validation loss curves for the MLP model under different regularization hyperparameters.

In general, as shown in Figure 3, the application of different data augmentation techniques slightly improved the final results. On the training set, we observed very good performance when using the Sobel filter, which enhances edge information in the images; however, this led to higher loss on the validation set, indicating poor generalization. Gaussian blur and rotation achieved similar effects on both training and validation sets, with rotation providing the best overall performance.

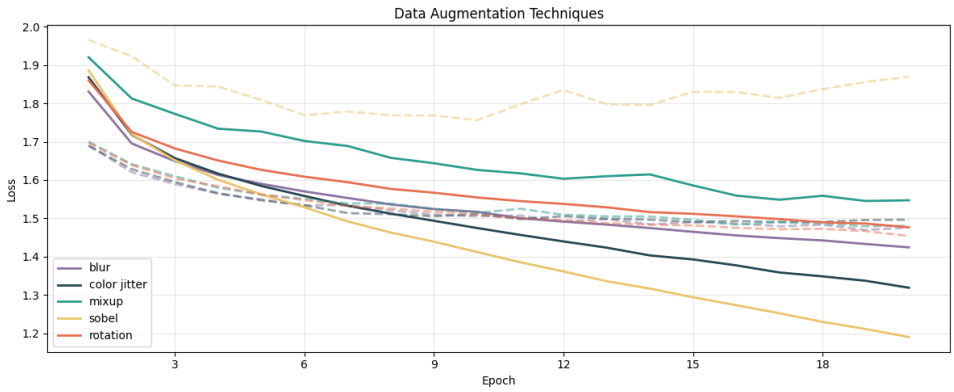


Figure 3: Training and validation loss curves for the MLP model using different data augmentation techniques.

Finally, Table 1 summarizes all experiments conducted for the MLP architecture, including their configurations as well as validation and test accuracies, sorted by test accuracy.

Table 1: Summary of MLP experiments with different hyperparameter configurations. Results are sorted by test accuracy.

LR	Optimizer	Dropout	Weight Decay	Augmentation	Val Acc	Test Acc
0.010	SGD+M	0.3	1×10^{-2}	-	0.3658	0.3675
0.010	Adam	0.3	1×10^{-4}	-	0.3777	0.3782
0.010	SGD+M	0.3	1×10^{-5}	sobel	0.3876	0.3848
0.001	SGD	0.3	1×10^{-4}	-	0.3875	0.3880
0.010	SGD	0.3	1×10^{-4}	-	0.4532	0.4543
0.010	SGD+M	0.1	1×10^{-4}	-	0.4576	0.4571
0.010	SGD+M	0.5	1×10^{-4}	mixup	0.4581	0.4572
0.100	SGD+M	0.3	1×10^{-5}	color_jitter	0.4663	0.4647
0.010	SGD+M	0.3	1×10^{-5}	-	0.4702	0.4651
0.010	SGD+M	0.3	1×10^{-4}	-	0.4691	0.4691
0.100	SGD	0.3	1×10^{-4}	-	0.4697	0.4692
0.010	SGD+M	0.3	1×10^{-5}	gaussian_blur	0.4689	0.4699
0.010	SGD+M	0.3	1×10^{-5}	-	0.4702	0.4703
0.010	SGD+M	0.3	1×10^{-5}	rotation	0.4768	0.4736

3.1.2 Convolutional Neural Networks (CNN)

The next model we evaluated was a Convolutional Neural Network (CNN) designed to process spatial image structures. The architecture consisted of four convolutional blocks with increasing channel sizes ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$). Each block included two sequences of Convolution, Batch Normalization, and ReLU activation, followed by MaxPooling. Feature aggregation was performed using Adaptive Average Pooling, and the classification head consisted of a fully connected layer ($256 \rightarrow 10$) with Dropout. During hyperparameter tuning, we first focused on the learning rate. The best performance after 20 training epochs was achieved with a learning rate of 0.01. In terms of optimization, the best results were obtained using the SGD optimizer with momentum. These results of the loss function for both experiments are illustrated in Figure 4.

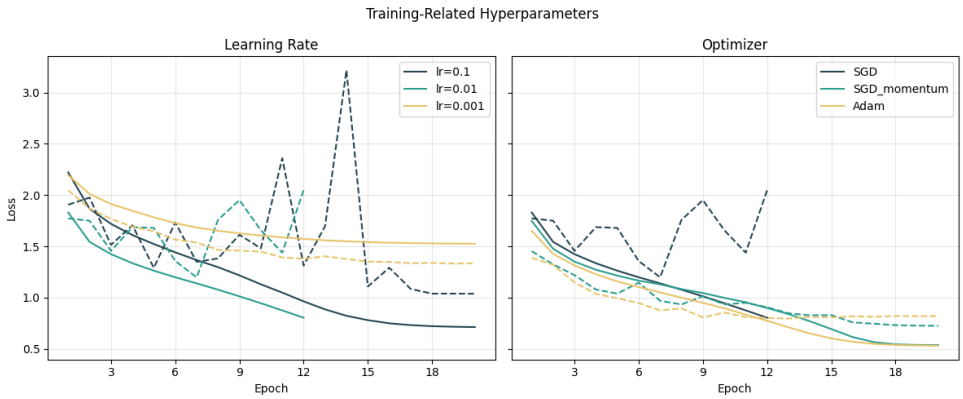


Figure 4: Training and validation loss curves for the CNN model under different training hyperparameters.

Regarding regularization-related hyperparameters, the highest accuracy was achieved with a weight decay value of 0.001. In contrast, the impact of Dropout was minimal, as all tested configurations produced very similar results, with accuracy consistently around 78%. This suggests that Dropout did not significantly influence the model's performance in this setup, although it still provided stable and reliable results across experiments. The results of the loss function can be seen in Figure 5.

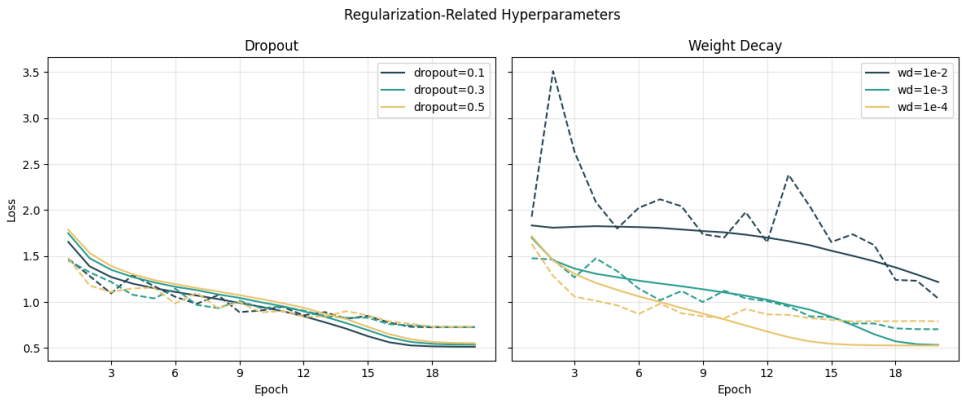


Figure 5: Training and validation loss curves for the CNN model under different regularization hyperparameters.

Finally, we examined the effect of data augmentation techniques in Figure 6. The inclusion of Sobel-based augmentation significantly degraded performance, reducing test accuracy to 59%. On the other hand, applying rotation-based augmentation led to the best overall performance among all tested configurations, achieving a test accuracy of 79%. This indicates that geometric transformations, such as rotation, were beneficial for improving the model's generalization.

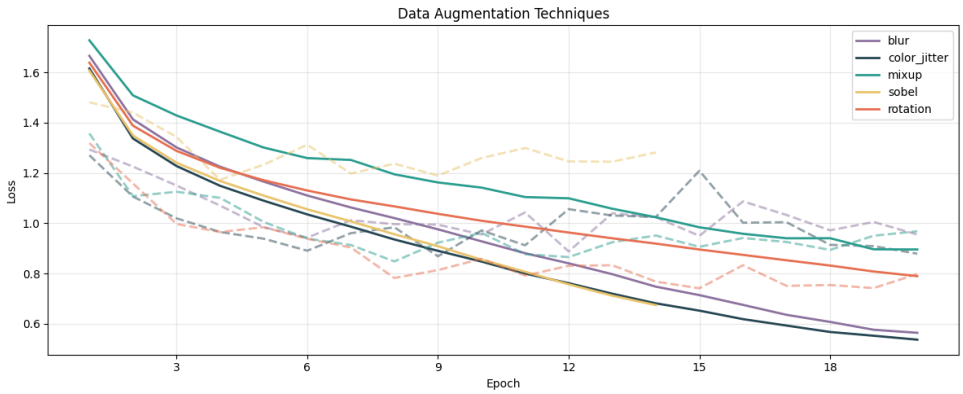


Figure 6: Training and validation loss curves for the CNN model using different data augmentation techniques.

All conducted CNN experiments, together with the accuracy results obtained on the validation and test sets, are presented in Table 2.

Table 2: Summary of CNN experiments with different hyperparameter configurations. Results are sorted by test accuracy.

LR	Optimizer	Dropout	Weight Decay	Augmentation	Val Acc	Test Acc
0.001	SGD	0.3	1×10^{-3}	-	0.5272	0.4705
0.100	SGD	0.3	1×10^{-3}	-	0.6672	0.5240
0.010	SGD+M	0.3	1×10^{-3}	sobel	0.5982	0.5957
0.010	SGD+M	0.3	1×10^{-2}	-	0.6572	0.6559
0.010	SGD	0.3	1×10^{-3}	-	0.4733	0.6663
0.010	SGD+M	0.3	1×10^{-3}	gaussian_blur	0.7313	0.7284
0.010	SGD+M	0.3	1×10^{-3}	color_jitter	0.7495	0.7471
0.010	Adam	0.3	1×10^{-3}	-	0.7616	0.7598
0.010	SGD+M	0.3	1×10^{-4}	-	0.7669	0.7637
0.010	SGD+M	0.3	1×10^{-3}	mixup	0.7716	0.7705
0.010	SGD+M	0.5	1×10^{-3}	-	0.7805	0.7792
0.010	SGD+M	0.3	1×10^{-3}	-	0.7831	0.7802
0.010	SGD+M	0.1	1×10^{-3}	-	0.7836	0.7821
0.010	SGD+M	0.3	1×10^{-3}	rotation	0.7942	0.7914

3.1.3 MobileNetV2

We decided to conduct experiments using pretrained models, and one of the architectures we selected was MobileNetV2. The training process involved a gradual unfreezing strategy. Initially, we kept 16 out of 19 layers frozen. After the first four epochs, we unfroze part of the network, leaving 12 layers still frozen. This approach allowed us to achieve relatively

good results.

When selecting training-related hyperparameters, we chose **a learning rate of 0.01**, which enabled fast convergence without leading to overfitting. We decided to use **SGD** as optimizer as it shows the best result without overfiting

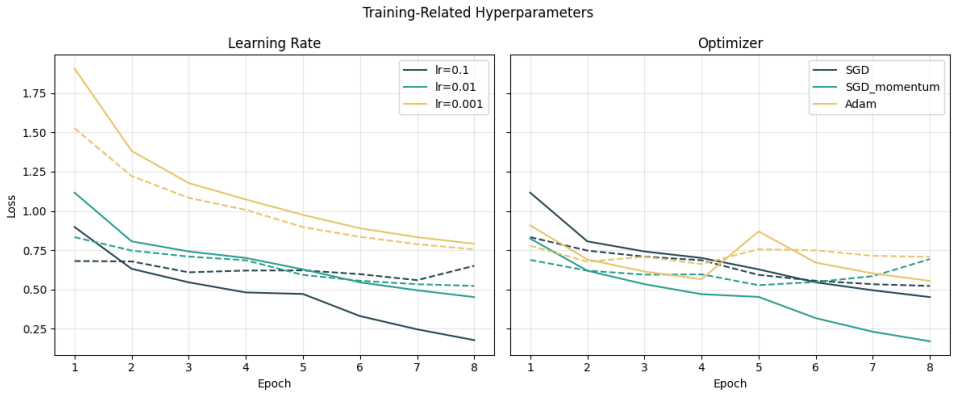


Figure 7: Training and validation loss curves for the MobileNetV2 model under different training hyperparameters.

The selection of regularization hyperparameters had little to no impact on the training process, as most tested values yielded similar results. The final configuration included **a dropout rate of 0.3** and **a weight decay of 1e-4**. The lack of noticeable differences may be attributed to the relatively large number of frozen layers.

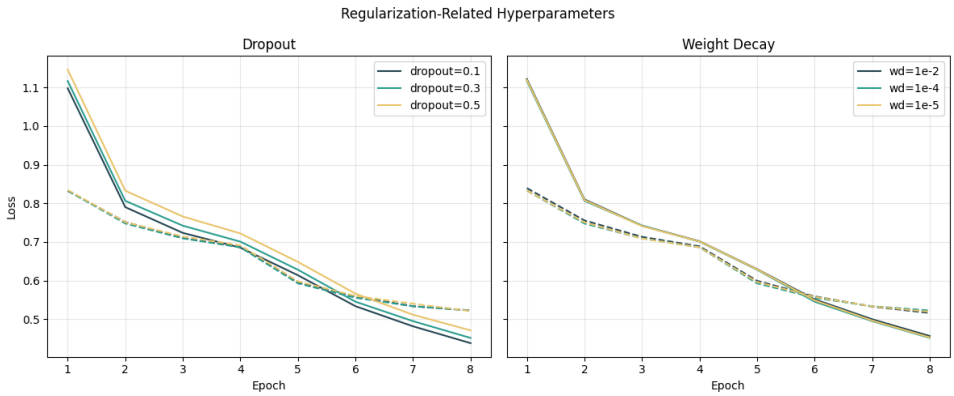


Figure 8: Training and validation loss curves for the MobileNetV2 model under different regularization hyperparameters.

Finally, we analyzed various data augmentation techniques. Most of them behaved similarly and did not significantly improve the model's performance. Gaussian blur, however, performed particularly poorly on both the validation and test sets.

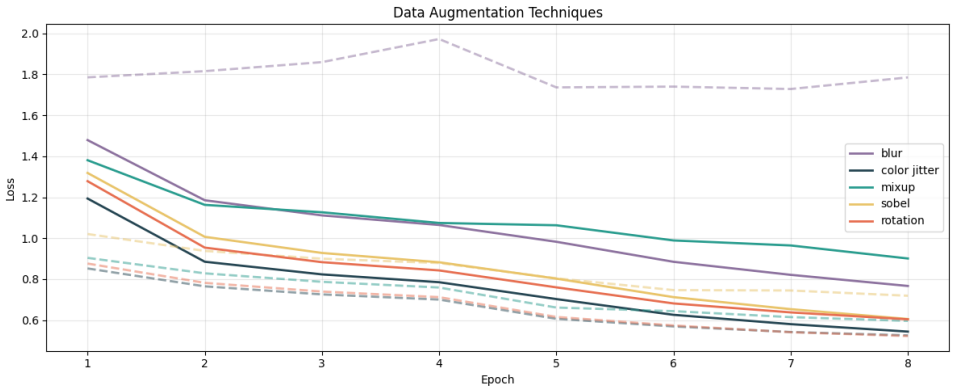


Figure 9: Training and validation loss curves for the MobileNetV2 model using different data augmentation techniques.

Finally, Table 3 summarizes all experiments conducted for the MobileNetV2 architecture, including their configurations as well as validation and test accuracies, sorted by test accuracy.

Table 3: Summary of MobileNetV2 experiments with different hyperparameter configurations. Results are sorted by test accuracy.

LR	**Optimizer**	**Dropout**	**Weight Decay**	**Augmentation**	**Val Acc**	**Test Acc**
0.010	SGD	0.3	1×10^{-4}	gaussian_blur	0.4200	0.4160
0.001	SGD	0.3	1×10^{-4}	-	0.7348	0.7325
0.010	SGD	0.3	1×10^{-4}	sobel	0.7440	0.7416
0.010	Adam	0.3	1×10^{-4}	-	0.7556	0.7551
0.010	SGD+M	0.3	1×10^{-4}	-	0.8061	0.8034
0.010	SGD	0.3	1×10^{-4}	mixup	0.8052	0.8047
0.100	SGD	0.3	1×10^{-4}	-	0.8080	0.8072
0.010	SGD	0.3	1×10^{-4}	rotation	0.8131	0.8103
0.010	SGD	0.3	1×10^{-4}	color_jitter	0.8119	0.8109
0.010	SGD	0.3	1×10^{-5}	-	0.8146	0.8113
0.010	SGD	0.3	1×10^{-4}	-	0.8134	0.8114
0.010	SGD	0.1	1×10^{-4}	-	0.8137	0.8125
0.010	SGD	0.5	1×10^{-4}	-	0.8140	0.8130
0.010	SGD	0.3	1×10^{-2}	-	0.8158	0.8142

3.1.4 ResNet-18

The final model we evaluated was ResNet-18, fine-tuned on the CINIC-10 dataset. The base model was initialized with pretrained weights and modified by replacing the final fully connected layer with a custom classification head consisting of Batch Normalization,

Dropout, and a Linear layer mapping to 10 classes. Additionally, a cosine annealing learning rate scheduler was applied during training, and mixed precision training with gradient scaling was used to improve efficiency. As shown in Figure 10, the loss function was analyzed across epochs (each experiment ran for 7 epochs). Similarly to previous experiments, we first investigated the impact of the learning rate and optimizer on model performance. The best results were achieved with a learning rate of 0.1. During optimizer tuning, we tested Adam with a lower learning rate (0.01), since it is known to perform better in such settings, while SGD and SGD with momentum were tested with a learning rate of 0.1. Both SGD with momentum and Adam achieved very good results, with a slight advantage observed for SGD with momentum.

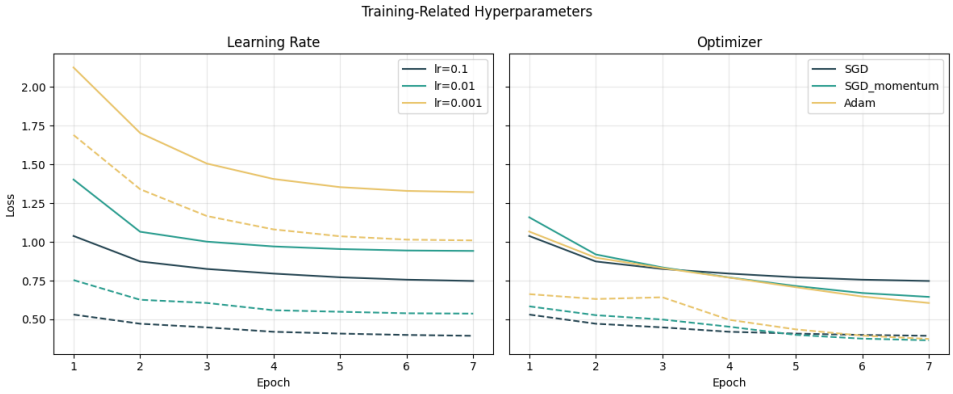


Figure 10: Training and validation loss curves for the ResNet-18 model under different training hyperparameters.

In Figure 11, we present the impact of regularization-related hyperparameters, specifically Dropout and weight decay, using a learning rate of 0.1 and the SGD with momentum optimizer. The results indicate that Dropout had minimal influence on performance, as no significant differences were observed across tested values. In contrast, weight decay had a more noticeable effect: using a relatively larger value (0.01) led to worse performance compared to smaller values such as 0.001 or 0.0001.

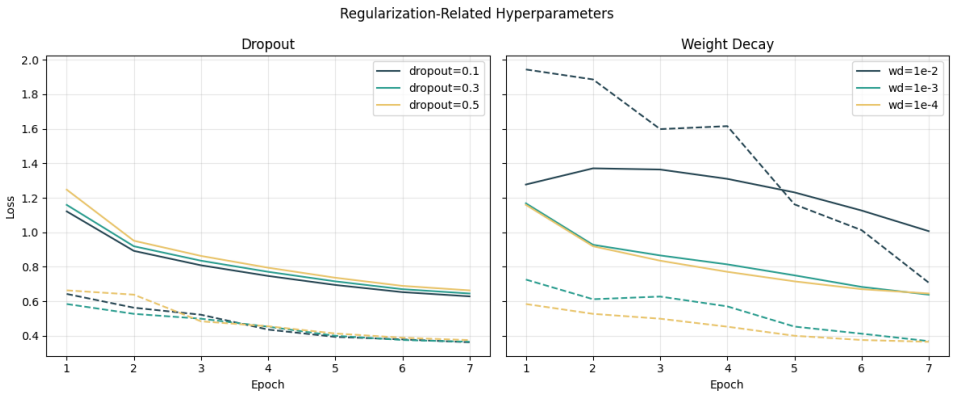


Figure 11: Training and validation loss curves for the ResNet-18 model under different regularization hyperparameters.

The effect of data augmentation techniques is illustrated in Figure 12. The inclusion of Gaussian blur augmentation significantly degraded performance, resulting in the worst accuracy among all experiments conducted for this model. On the other hand, applying color jitter augmentation led to the best overall performance. This confirms that while some augmentation techniques can improve generalization, others may negatively impact the model depending on the nature of the dataset and transformations applied.

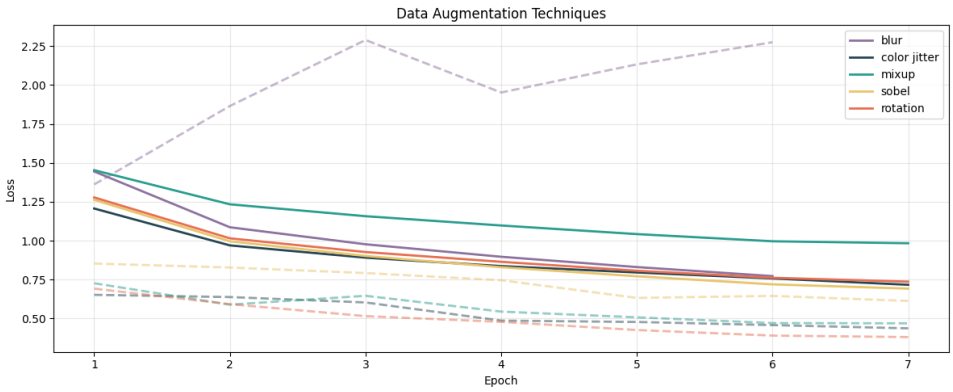


Figure 12: Training and validation loss curves for the ResNet-18 model using different data augmentation techniques.

Finally, Table 4 summarizes the accuracy results obtained on the validation and test sets for all conducted experiments with this model.

Table 4: Summary of ResNet-18 experiments with different hyperparameter configurations. Results are sorted by test accuracy.

LR	**Optimizer**	**Dropout**	**Weight Decay**	**Augmentation**	**Val Acc**	**Test Acc**
0.100	SGD+M	0.3	1×10^{-4}	gaussian_blur	0.2810	0.2749
0.001	SGD	0.3	1×10^{-4}	-	0.6787	0.6784
0.100	SGD+M	0.3	1×10^{-2}	-	0.7769	0.7738
0.100	SGD+M	0.3	1×10^{-4}	sobel	0.8079	0.8085
0.010	SGD	0.3	1×10^{-4}	-	0.8338	0.8326
0.100	SGD+M	0.3	1×10^{-4}	rotation	0.8878	0.8848
0.100	SGD	0.3	1×10^{-4}	-	0.8867	0.8853
0.100	SGD+M	0.3	1×10^{-4}	mixup	0.8912	0.8892
0.100	SGD+M	0.5	1×10^{-4}	-	0.8922	0.8906
0.001	Adam	0.3	1×10^{-4}	-	0.8928	0.8919
0.100	SGD+M	0.3	1×10^{-3}	-	0.8935	0.8930
0.100	SGD+M	0.3	1×10^{-4}	-	0.8955	0.8940
0.100	SGD+M	0.1	1×10^{-4}	-	0.8969	0.8941
0.100	SGD+M	0.3	1×10^{-4}	color_jitter	0.8943	0.8945

3.2 Subset Training

Before conducting few-shot learning experiments, we performed preliminary training on progressively smaller subsets of the CINIC-10 dataset. The goal of this experiment was to evaluate how different architectures handle extremely limited amounts of training data. By restricting the training set to 1%, 5%, and 10% of the full dataset, we can observe the model's ability to extract meaningful representations even when data is scarce.

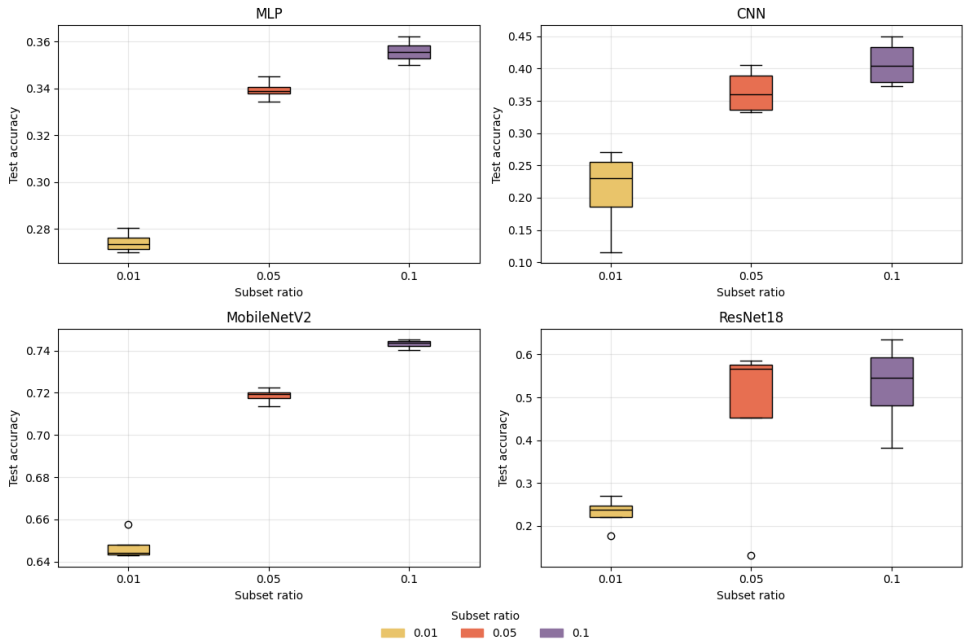


Figure 13: Test accuracy for models trained on 1%, 5% and 10% subset of train set.

Table 5 summarizes the test accuracies for four different models trained on varying dataset fractions. As expected, the performance of all models improves as more training data becomes available. MobileNetV2 consistently achieves the highest accuracy across all subsets, showing strong robustness to limited data. Standard CNN and MLP models improve gradually with larger subsets but remain significantly below MobileNetV2. ResNet18 shows more variability, particularly on the smallest subsets, which may be attributed to overfitting due to its larger capacity.

Table 5: Summary of the subset training experiment.

Subset	**Model**	**Test Acc**
1%	MLP	0.2743 ± 0.0046
	CNN	0.2113 ± 0.0687
	MobileNetV2	$\mathbf{0.6472 \pm 0.0071}$
	ResNet18	0.2303 ± 0.0391
5%	MLP	0.3393 ± 0.0044
	CNN	0.3646 ± 0.0358
	MobileNetV2	$\mathbf{0.7187 \pm 0.0037}$
	ResNet18	0.4621 ± 0.2212
10%	MLP	0.3558 ± 0.0053
	CNN	0.4079 ± 0.0372
	MobileNetV2	$\mathbf{0.7431 \pm 0.0021}$
	ResNet18	0.5278 ± 0.1084

3.3 Few-Shot Learning

Few-shot learning (FSL) is a paradigm in machine learning that aims to train models capable of generalizing to new classes given only a small number of labeled examples. Unlike traditional supervised learning, where models require large amounts of labeled data per class, FSL focuses on learning transferable representations that can be reused across different tasks.

A standard evaluation framework in few-shot learning is the N-way K-shot classification setting. In this setup:

- N denotes the number of classes in a given task,
- K denotes the number of labeled examples (shots) available per class.

Each task (also called an episode) is divided into two parts:

- The **support set**, which contains K labeled examples for each of the N classes and is used to construct class representations.
- The **query set**, which contains unseen examples from the same N classes and is used to evaluate the model's ability to generalize.

During training, the model is exposed to many such episodes, each involving different subsets of classes. This episodic training strategy encourages the model to learn a general embedding space in which samples from the same class are close together, while samples from different classes are far apart.

In this project, few-shot learning was tested with Prototypical Networks. The dataset was split into training and test classes, so the model was evaluated on unseen classes. For each episode, N classes were selected, K examples per class were used as support, and Q examples per class were used as query. The model learned image embeddings and classified query samples by comparing them to class prototypes.

We tested several N-way K-shot settings: N = 2, 3, 4 and K = 1, 5, 10. For each setting, the model was trained for a fixed number of episodes and then evaluated on the test set.

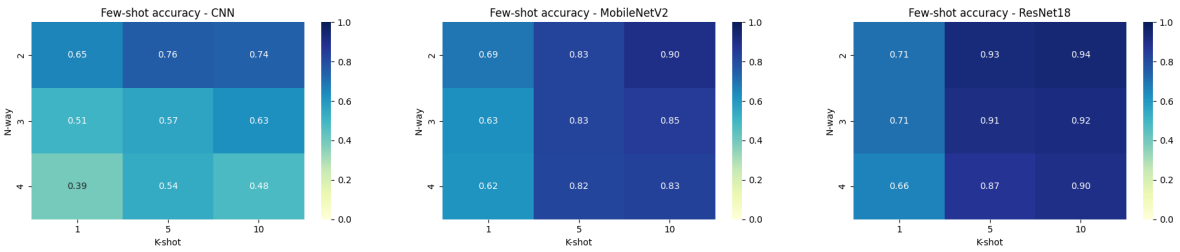
Table 6 presents the classification accuracy obtained for different model architectures under the N-way K-shot evaluation protocol.

We can observe that increasing the number of support samples (K) consistently improves performance. For example, in the case of ResNet18 for N = 2, accuracy increases from 0.7050 in the 1-shot setting to 0.9385 in the 10-shot setting. This behavior is expected, as more examples per class allow the model to construct more reliable class prototypes. Increasing the number of classes (N) makes the task more challenging. This is visible across all architectures, where accuracy decreases as N increases. For instance, the CNN model achieves 0.6540 for N = 2, K = 1, but only 0.3915 for N = 4, K = 1. This reflects the increased difficulty of distinguishing between a larger number of classes in each episode.

Table 6: Few-shot classification accuracy (N-way K-shot) for different model architectures. Results are averaged over episodic evaluations.

Model		**N = 2**	**N = 3**	**N = 4**
CNN	K = 1	0.6540	0.5060	0.3915
	K = 5	0.7555	0.5653	0.5393
	K = 10	0.7425	0.6260	0.4790
MobileNetV2	K = 1	0.6940	0.6270	0.6153
	K = 5	0.8310	0.8290	0.8218
	K = 10	0.9020	0.8517	0.8328
ResNet18	K = 1	0.7050	0.7060	0.6568
	K = 5	0.9260	0.9063	0.8713
	K = 10	0.9385	0.9153	0.8985

The results are visualized in the form of heatmaps in Figure 14.



(a) CNN (b) MobileNetV2 (c) ResNet18

Figure 14: Comparison of few-shot classification performance across different model architectures.

4 Conclusions

Summarizing the best results in Table 7 obtained for all tested models, it can be clearly seen that the MLP model achieves the worst performance, while ResNet-18 performs the best. CNN and MobileNetV2 obtain similar results, with a slight advantage for MobileNetV2.

Looking at the hyperparameters, most of the best-performing models use the SGD optimizer with momentum, which suggests that this optimization method works well for this task. Additionally, a dropout value of 0.3 appears to be a good choice, as it is present in almost all top configurations.

Another important observation is the role of data augmentation. In three out of four best models, augmentation techniques were used during training. This indicates that applying transformations to the training data helps improve generalization and overall performance.

Table 7: Summary of the best-performing configurations for each model.

Model	**LR**	**Optimizer**	**Dropout**	**Weight Decay**	**Augmentation**	**Test Acc**
MLP	0.01	SGD+M	0.3	1×10^{-5}	rotation	0.4736
CNN	0.01	SGD+M	0.3	1×10^{-3}	rotation	0.7914
MobileNetV2	0.01	SGD	0.3	1×10^{-2}	-	0.8142
ResNet18	0.10	SGD+M	0.3	1×10^{-4}	color_jitter	0.8945

For the few-shot experiments, the results demonstrate that both MobileNetV2 and ResNet-18 substantially outperform simpler models like CNN and MLP when only a few labeled examples per class are available. Accuracy consistently improves with higher K-shot settings and fewer N-way classes, highlighting the models' ability to quickly adapt to new classes. These findings confirm that pretrained architectures with strong feature extraction capabilities are particularly effective in low-data, few-shot scenarios.

References

[1] K. He et al. "Deep Residual Learning for Image Recognition". In: *arXiv preprint arXiv:1512.03385* (2015). DOI: 10.48550/arXiv.1512.03385.

[2] A.G. Howard et al. "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications". In: *arXiv preprint arXiv:1704.04861* (2017). DOI: 10.48550/arXiv.1704.04861.

[3] G. Huang et al. "Densely Connected Convolutional Networks". In: *IEEE Conference on Computer Vision and Pattern Recognition* (2016). DOI: 10.48550/arXiv.1608.06993.

[4] Y. LeCun et al. "Backpropagation Applied to Handwritten Zip Code Recognition". In: *Neural Computation* 1.4 (1989), pp. 541–551. DOI: 10.1162/neco.1989.1.4.541.

[5] F. Rosenblatt. "The perceptron: A probabilistic model for information storage and organization in the brain". In: *Psychological Review* 65.6 (1958), pp. 386–408. DOI: 10.1037/h0042519.

[6] K. Simonyan and A. Zisserman. "Very Deep Convolutional Networks for Large-Scale Image Recognition". In: *CoRR* (2014). DOI: 10.48550/arXiv.1409.1556.